

Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 04

Conversión de Decimal a Binario en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: 23 de mayo de 2026

Índice

1. Introducción	2
2. Organización del programa	2
3. Código fuente	4
3.1. Código C++ (main.cpp)	4
3.2. Código ensamblador (conversion.asm)	4
4. Instrucciones x86 utilizadas	6
5. El rol de los registros	6
6. Ejemplo de ejecución	7
7. Repositorio GitHub	8
8. Conclusiones	8

1. Introducción

Esta práctica consiste en escribir un programa que combine **C++** y **lenguaje ensamblador x86**: C++ se encarga de la interacción con el usuario (leer el número y mostrar el resultado), mientras que una función escrita en ensamblador realiza la conversión del número decimal a su representación en binario, operando directamente sobre los bits del valor.

El objetivo principal no es simplemente “convertir un número”, sino entender cómo el procesador manipula los datos en su forma más elemental, bit por bit. Para lograrlo, es necesario comprender dos conceptos fundamentales: la **representación binaria de los números** y las **operaciones de desplazamiento de bits**.

¿Qué es la representación binaria?

Las computadoras almacenan y procesan todos sus datos en **sistema binario**: únicamente con los dígitos 0 y 1. Cada dígito binario se llama *bit*. Un número entero de 32 bits es simplemente una secuencia de 32 ceros y unos, donde cada posición representa una potencia de 2:

$$45_{10} = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 101101_2 \quad (1)$$

¿Qué es un desplazamiento de bits (SHL)?

La instrucción **SHL** (*Shift Left*) mueve todos los bits de un registro un lugar hacia la izquierda. El bit que “se cae” por el extremo izquierdo no desaparece: el procesador lo atrapa en una bandera interna llamada **Carry Flag (CF)**, que puede leerse en el siguiente ciclo. Esta es la mecánica que permite extraer los bits de un número uno por uno, de mayor a menor.

2. Organización del programa

El proyecto consta de dos archivos que trabajan en conjunto:

Archivo	Qué hace
main.cpp	Pide al usuario un número decimal, llama a la función ensamblador y muestra el resultado binario en pantalla.
conversion.asm	Recibe el número entero, localiza su bit más significativo, extrae cada bit de mayor a menor y construye la cadena de texto binaria en un buffer de memoria.

Cuadro 1: Archivos del proyecto

El programa se divide internamente en cuatro fases:

- **Preservar registros:** antes de usar EBX y EDI, sus valores se guardan en la pila para no interferir con C++.
- **Caso especial (cero):** si el número es 0, se escribe directamente '0' en el buffer y se termina.
- **Localizar el MSB:** la instrucción BSR encuentra la posición del bit más significativo; el número se alinea a la izquierda con SHL para preparar la extracción.
- **Ciclo de escritura:** en cada vuelta, SHL extrae un bit hacia la *Carry Flag*; SETC lo lee y ADD lo convierte al carácter '0' o '1'.

El flujo general del programa es el siguiente:

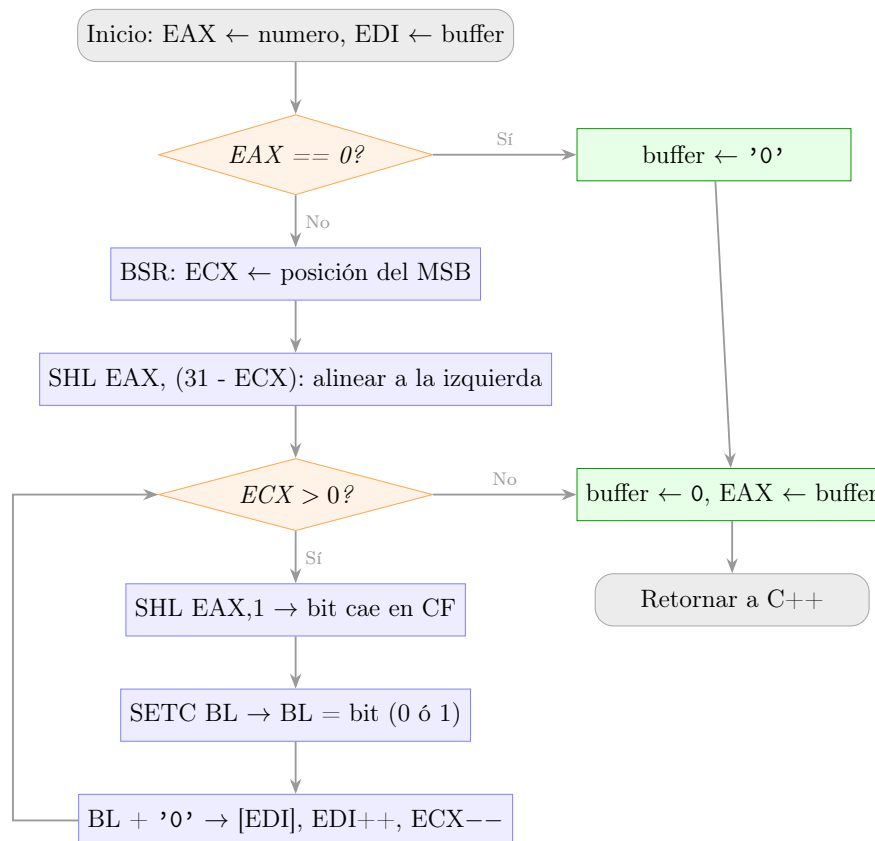


Figura 1: Diagrama de flujo del algoritmo de conversión decimal a binario

1. C++ lee el número del usuario y llama a `decimalABinario` pasándolo como parámetro.
2. El ensamblador verifica si el número es cero (caso especial).
3. BSR localiza el bit más alto; SHL alinea el número para que ese bit quede en el extremo izquierdo de EAX.

4. El ciclo extrae bit por bit con `SHL`: cada bit que “se derrama” por la izquierda es capturado por `SETC` y escrito como `'0'` o `'1'` en el buffer.
5. Al terminar el ciclo, se coloca el terminador de cadena y se retorna la dirección del buffer a `C++`.
6. `C++` recibe el puntero e imprime la cadena resultante.

3. Código fuente

3.1. Código C++ (main.cpp)

```
1 #include <iostream>
2 using namespace std;
3
4 extern "C" char* decimalABinario(int numero);
5
6 int main() {
7     int decimal;
8
9     cout << "Ingresa un numero decimal: ";
10    cin >> decimal;
11
12    char* binario = decimalABinario(decimal);
13
14    cout << "DEC: " << decimal << endl;
15    cout << "BIN: " << binario << endl;
16
17    return 0;
18 }
```

Listing 1: main.cpp — Interfaz con el usuario

3.2. Código ensamblador (conversion.asm)

```
1 .586
2 .model flat, C
3
4 .data
5     buffer BYTE 33 DUP(0) ; 32 bits + terminador nulo
6
7 .code
8
9 decimalABinario PROC numero:DWORD
10
11     push ebx ; Guardamos EBX en la pila
12     push edi ; Guardamos EDI en la pila
13
14     mov eax, numero ; EAX = numero a convertir
15     lea edi, buffer ; EDI apunta al buffer de salida
```

```

16
17     test    eax, eax                ; ?El numero es cero?
18     jz      caso_base              ; Si es cero, caso especial
19
20     bsr     ecx, eax                ; ECX = posicion del bit mas
        significativo
21
22     mov     edx, 31
23     sub     edx, ecx                ; EDX = cuantos lugares desplazar
24     push    ecx                    ; Guardamos ECX (SHL sobreescribira CL)
25     mov     cl, dl                  ; CL = lugares a desplazar
26     shl     eax, cl                ; Alineamos el MSB al extremo izquierdo
27     pop     ecx                    ; Recuperamos el contador de bits
28     inc     ecx                    ; +1 para incluir el MSB en el ciclo
29
30 escribir_bits:
31     shl     eax, 1                  ; El bit mas alto cae a la Carry Flag
32     setc    bl                      ; BL = 1 si CF=1, BL = 0 si CF=0
33     add     bl, '0'                 ; Convertimos a caracter '0' o '1'
34     mov     BYTE PTR [edi], bl      ; Escribimos en el buffer
35     inc     edi                     ; Avanzamos el puntero
36     dec     ecx                    ; Un bit menos
37     jnz     escribir_bits           ; Repetir si quedan bits
38     jmp     fin
39
40 caso_base:
41     mov     BYTE PTR [edi], '0'      ; Escribimos '0' directamente
42     inc     edi                     ; Avanzamos el puntero
43
44 fin:
45     mov     BYTE PTR [edi], 0        ; Terminador de cadena
46     lea     eax, buffer              ; Retornamos la direccion del buffer
47     pop     edi
48     pop     ebx
49     ret
50
51 decimalABinario ENDP
52
53 END

```

Listing 2: conversion.asm — Conversión de decimal a binario

4. Instrucciones x86 utilizadas

Instrucción	Qué hace en términos simples
MOV	Copia un valor de un lugar a otro (entre registro y memoria).
LEA	Carga en un registro la <i>dirección</i> de una variable, no su contenido.
TEST	Compara un valor consigo mismo sin modificarlo; activa la bandera de cero si el resultado es 0.
JZ	Salta si la bandera de cero está activa (el valor comparado era 0).
JNZ	Salta si la bandera de cero <i>no</i> está activa (el valor no era 0).
JMP	Salta incondicionalmente a otra parte del código.
BSR	Busca de derecha a izquierda el primer bit en 1 y guarda su posición.
SHL	Desplaza todos los bits hacia la izquierda; el bit que “se cae” queda en la <i>Carry Flag</i> .
SETC	Escribe 1 en un registro si la <i>Carry Flag</i> está activa; escribe 0 si no.
ADD	Suma dos valores y guarda el resultado en el primero.
INC	Incrementa en 1 el valor del operando.
DEC	Decrementa en 1 el valor del operando.
PUSH	Guarda un valor en la pila de memoria.
POP	Recupera el último valor guardado en la pila.
RET	Regresa el control al código que llamó a esta función.

Cuadro 2: Instrucciones x86 utilizadas en el programa

5. El rol de los registros

En este programa intervienen varios registros del procesador, cada uno con una responsabilidad específica:

- **EAX** (*Extended Accumulator Register*): contiene el número a convertir. A medida que el ciclo avanza, sus bits van siendo extraídos uno por uno mediante desplazamientos.
- **ECX** (*Extended Counter Register*): actúa como contador del ciclo. Se inicializa con la posición del MSB más uno, y se decrementa en cada iteración hasta llegar a cero.
- **EDX** (*Extended Data Register*): almacena temporalmente la cantidad de lugares a desplazar para la alineación inicial del número ($31 - \text{posición_MSB}$).

- **EBX / BL**: BL (byte bajo de **EBX**) recibe el bit extraído vía **SETC** y lo convierte al carácter de texto correspondiente antes de escribirlo en el buffer.
- **EDI** (*Extended Destination Index*): actúa como puntero de escritura sobre el buffer. Avanza un byte en cada iteración.
- **CL**: byte bajo de **ECX**; es el único registro que **SHL** acepta como operando de desplazamiento variable.

La variable **buffer** se declara en **.data** con 33 bytes inicializados en 0: 32 para los dígitos binarios y 1 para el terminador de cadena que C++ necesita para saber dónde termina la cadena.

6. Ejemplo de ejecución

Para el número decimal **45**:

Iteración	Bit extraído (CF)	BL	Carácter escrito	ECX restante
1	1	1	'1'	5
2	0	0	'0'	4
3	1	1	'1'	3
4	1	1	'1'	2
5	0	0	'0'	1
6	1	1	'1'	0

Cuadro 3: Extracción bit a bit de $45 = 101101_2$

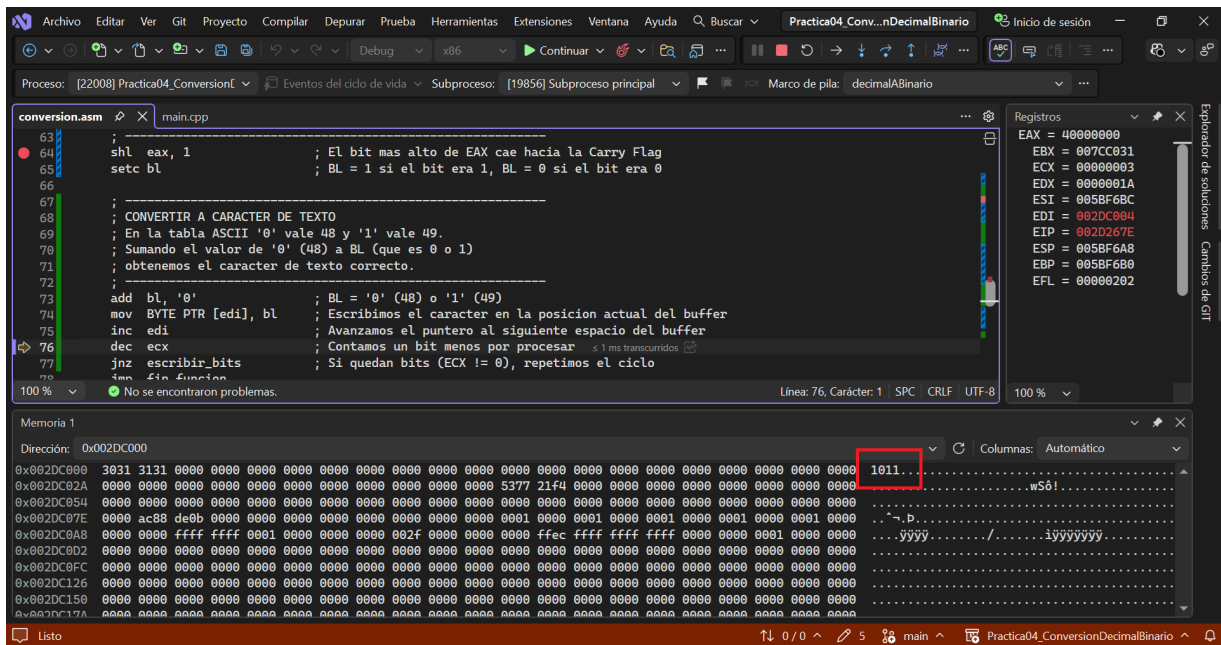


Figura 2: Ventana Memory de Visual Studio: buffer con el resultado "101101" tras convertir 45.

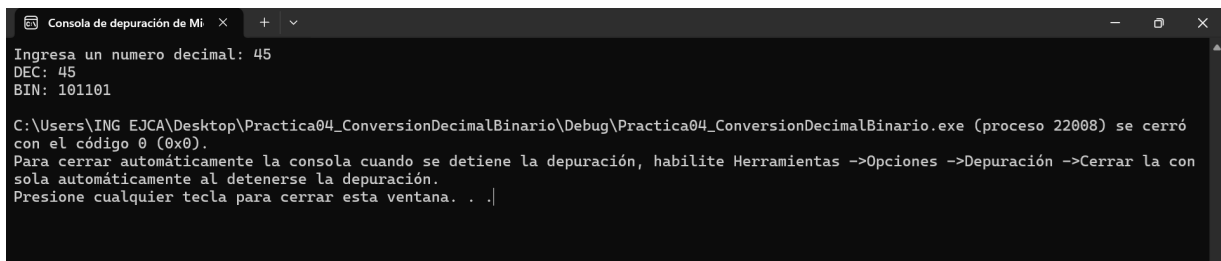


Figura 3: Salida en consola: el usuario ingresa 45 y el programa devuelve 101101.

7. Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Practica04_ConversionDecimalBinario

8. Conclusiones

- El programa demuestra que **convertir a binario es leer los bits de un número uno por uno**: la computadora no “calcula” la representación binaria, simplemente expone los bits que ya estaban ahí desde el principio.
- La instrucción BSR permite encontrar el bit más significativo en un solo paso de hardware, evitando ceros innecesarios a la izquierda sin necesidad de un ciclo adicional.

- El uso combinado de **SHL** y **SETC** es un patrón idiomático en ensamblador para extraer bits de forma secuencial: cada desplazamiento a la izquierda “derrama” un bit hacia la *Carry Flag*, donde puede leerse directamente.
- La **interfaz entre C++ y ensamblador** mediante `extern "C"` ilustra cómo los lenguajes de alto nivel pueden delegar operaciones críticas de bajo nivel a rutinas escritas directamente en ensamblador, devolviendo resultados a través de registros o punteros.
- Esta práctica evidencia que lo que un lenguaje como Python hace con `bin(45)` produce exactamente la misma secuencia de bits que este programa extrae manualmente: la diferencia es que aquí el proceso es completamente visible y controlado por el programador.